

Práctica 2: PSD DE SEÑALES ALEATORIAS (GNURADIO)

Silvia Fernanda León Rodríguez; Silvia2215738@correo.uis.edu.co

José Manuel Serrano Sarmiento; Jose2201866@correo.uis.edu.co

Laura Fernanda Vilbao Parra; Laura2205123@correo.uis.edu.co

https://github.com/silviafernandaleon12-dot/ComII_LabA1B

Abstract

This laboratory focused on the analysis of the Power Spectral Density (PSD) of random signals using GNU Radio as a software-defined radio (SDR) platform. Since many real-world phenomena can be represented as random processes, understanding their spectral behavior is essential in communication systems. During the experiment, random binary sequences, white noise, audio signals, and image data were generated and examined. Parameters such as bit rate, sampling frequency, and samples per symbol (Sps) were modified to evaluate their influence on the signal behavior in both the time and frequency domains. The obtained results demonstrate the usefulness of PSD for characterizing signals and emphasize its importance in the design, analysis, and optimization of modern digital communication systems.

1. INTRODUCCIÓN

La densidad espectral de potencia (PSD) es una herramienta ampliamente utilizada en el análisis de señales, ya que permite estudiar cómo se distribuye la potencia de una señal en función de la frecuencia. Su importancia radica en que proporciona información sobre el contenido espectral de las señales y facilita la comprensión de fenómenos presentes en sistemas de comunicaciones digitales.

En esta práctica se empleó GNU Radio como plataforma de Radio Definida por Software (SDR) para generar y analizar diferentes tipos de señales. A través de los experimentos realizados fue posible observar el efecto que tienen parámetros como la tasa de bits (Rb), la frecuencia de muestreo (fs), las muestras por símbolo (Sps), el ancho de banda (BW) y la interpolación sobre el comportamiento temporal y espectral de las señales.

Además del análisis de secuencias binarias aleatorias, se estudiaron señales de ruido blanco y señales reales obtenidas a partir de imágenes y archivos de audio. Esto permitió relacionar los conceptos teóricos de PSD con aplicaciones prácticas en sistemas modernos de procesamiento y transmisión de información.

2. METODOLOGÍA EXPERIMENTAL

Con el fin de cumplir con los objetivos planteados en esta práctica, se trabajó con un flujograma de GNURadio dado por la guía [2], el cual me permite hacer un análisis detallado de la frecuencia y la potencia de una señal binaria bipolar aleatoria, donde se ejecutó los siguientes pasos: Generación de la señal binaria bipolar aleatoria, cálculo de su espectro en frecuencia, densidad espectral de potencia y variación del parámetro sps. La señal se genera siguiendo el diagrama de flujo, mostrado en la figura 2.

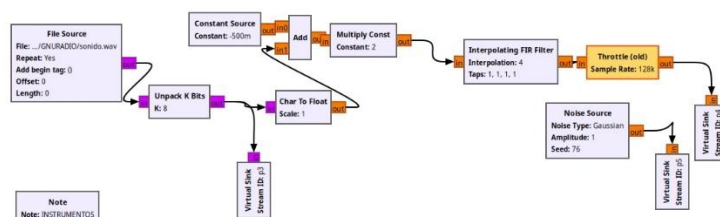


Figura 1. Generación de señales binarias aleatorias.

Creación de un ruido blanco para observar su comportamiento en el tiempo y en frecuencia. Esto se logra mediante el uso del bloque Noise Source de GNURadio, seleccionando un ruido de tipo gaussiano. La gráfica de la PSD y el espectro se realiza con el sistema de bloques presentado en la siguiente figura.

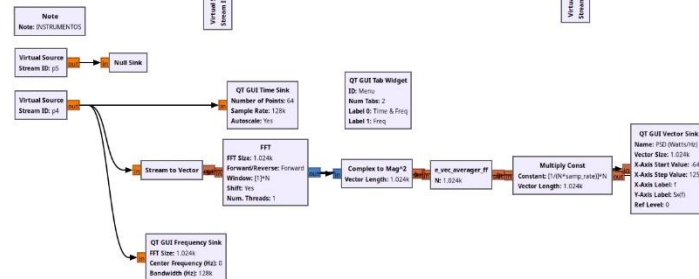


Figura 2. Ruido blanco

Se Utilizó una señal proveniente del mundo real, cambiando el bloque de entrada, Random Source por el conjunto de bloques mostrado a continuación, los archivos a utilizar fueron de audio e imagen.

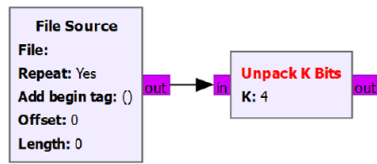


Figura 3. Bloques utilizados.

Se analizan los diferentes bloques del flujograma y se determina que función cumple dentro del sistema, también se analiza cada parámetro y como estos me cambian la PSD.

3. PROCEDIMIENTO

Se analizó la señal binaria aleatoria generada, donde se varía el parámetro Sps, obteniendo una señal con su respectiva PSD como la mostrada en la siguiente figura.

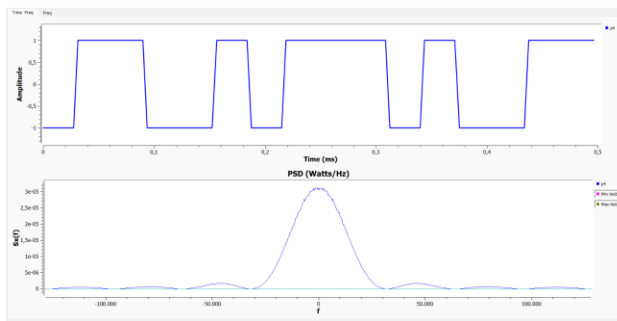


Figura 4. Gráfica de una señal binaria bipolar aleatoria en el tiempo y su respectiva PSD.

Los resultados obtenidos al variar el parámetro Sps de la señal binaria bipolar aleatoria son los siguientes:

sps	Tasa de Bits (Rb)	Frecuencia de Muestreo (fs)	Ancho de Banda (BW)
1	32 kbps	32 kHz	16 kHz
4	32 kbps	128 kHz	16 kHz
8	32 kbps	256 kHz	16 kHz
16	32 kbps	512 kHz	16 kHz

Tabla 1. Datos obtenidos.

En la tabla se presenta un análisis detallado de cómo, al variar las muestras por segundo (sps), la señal bipolar aleatoria experimenta cambios en su frecuencia de muestreo. Dado que se tiene una frecuencia de muestreo de 32 kHz y una tasa de bits de 32 kbits/s, se utiliza la siguiente fórmula para cálculo el número de lóbulos en la PSD de la señal binaria aleatoria rectangular:

$$N_{\text{lóbulos}} = \frac{2f_s}{Sps \cdot Rb}$$

Con esta fórmula calculamos los lóbulos para cada SPS:

Sps	Frecuencia de muestreo (fs)	Lóbulos (N_lóbulos)
2	32 kHz	32
4	128 kHz	16
8	256 kHz	8
16	512 kHz	4

Se analizó también el ruido blanco, que se obtiene del bloque

de ruido gaussiano y se obtuvo la siguiente gráfica:

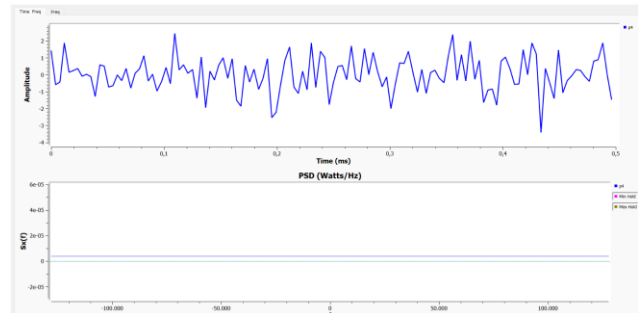


Figura 5. Ruido blanco.

Para las señales extraídas del mundo real, en este caso el audio y la imagen, las gráficas son las siguientes:

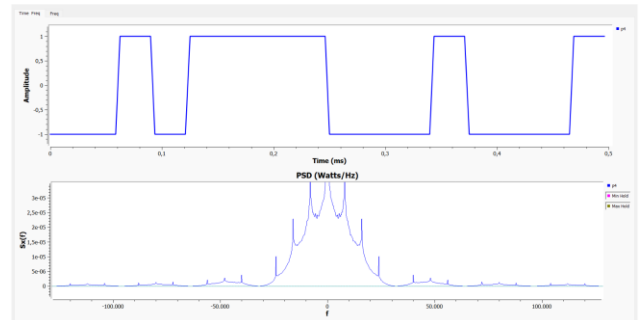


Figura 6. Señal de archivo de sonido.

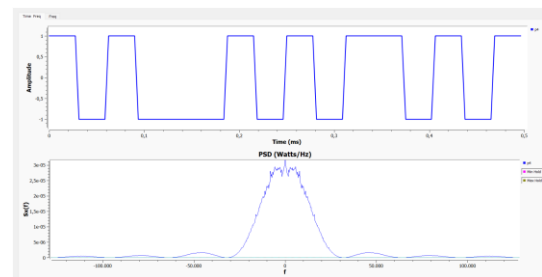


Figura 7. Señal de archivo de imagen.

Se hicieron cambios en el parámetro h, tal que la forma de tomar muestras por símbolo permite obtener distintos tipos de señal, una de estas es la observada a continuación:

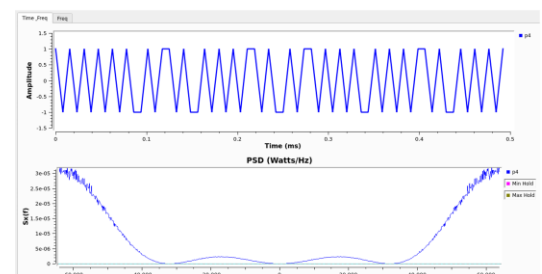


Figura 8. Señal aleatoria modificada para obtener la forma de dientes de sierra.

También se logró observar que al modificar el off set, se obtienen formas que representan modulaciones ook o bpsk, usando como base esta señal aleatoria, esto lo se puede

observar en las siguientes gráficas:

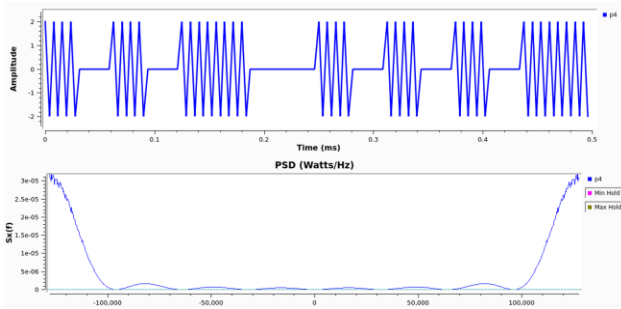


Figura 9. Modulación OOK obtenida.

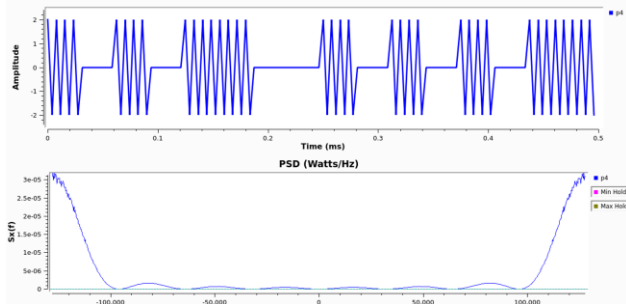


Figura 10. Modulación BPSK obtenida.

4. ANÁLISIS DE RESULTADOS

El parámetro de muestras por símbolo (sps) afecta directamente en la calidad de interpolación de la señal binaria bipolar aleatoria, ya que, cuando este valor es pequeño la señal de salida resultante tiene cambios abruptos entre los símbolos transmitidos debido a que no tiene suficientes repeticiones de un símbolo para suavizar ese cambio, distinto a cuando se tiene un valor grande de sps.

Así pues, dicho parámetro se inserta en el bloque Interpolating FIR Filter, el cual se utiliza para aumentar la tasa de muestreo de una señal insertando muestras interpoladas entre las originales. Se ajusta el parámetro Interpolation igual a Sps para igualar al número de muestras necesarias por símbolo. Se llega además a la conclusión de que trabajar con la señal p3 no tiene mucho sentido, dado que esta es la señal antes de pasar por el bloque de interpolación, y por tanto, es más difícil de interpretar. Finalmente, se infiere que la relación de frecuencias luego del bloque de interpolación es como la presentada en la siguiente ecuación.

$$f_{s,p3} = \frac{f_{s,p4}}{SPS}$$

Se analizó una señal real de ruido blanco, confirmando que mantiene potencia constante en todo el rango observado en GNU Radio, aunque no puede evidenciarse un ancho de banda infinito por las limitaciones del software. El bloque **Throttle** regula la velocidad de muestreo según la tasa de

bits (Rb) y el número de símbolos (Sps), evitando sobrecargar el sistema. Cuando la señal p4 toma valores de 0 y 1, se vuelve unipolar y aparece una componente DC en 0 Hz. En la práctica, el ancho de banda queda limitado por las capacidades del sistema, por lo que la PSD se ve plana solo en un rango definido. Finalmente, al aplicar interpolación, la señal gana resolución, lo que mejora el muestreo y aproxima mejor su comportamiento teórico.

La resolución espectral (RE) del analizador de espectros se puede calcular si se conoce la frecuencia de muestreo (f_s) y el número de puntos de la FFT (N), a través de la siguiente ecuación:

$$RE = \frac{f_s}{N}$$

Al aumentar el parámetro **K** del bloque *Unpack K Bits* a 16, la señal se agrupa en bloques de 16 bits, generando en la PSD impulsos distribuidos por el espectro debido a armónicos y mayor dispersión de energía, lo que también incrementa el ancho de banda. La frecuencia de muestreo necesaria para capturar todos los lóbulos se calcula como:

$$f_s = SPS * RB$$

El bloque “*Char to float*” no cambia la frecuencia de muestreo, es decir, la frecuencia de muestreo de la entrada será la misma frecuencia de muestreo en la salida. Esto se entiende al conocerse que el bloque solo se usa para cambiar caracteres o datos de tipo char a tipo float (punto flotante) con el fin de tener una mejor precisión en las siguientes etapas del sistema. Se hizo la variación del parámetro **h** correspondiente a un vector que tiene la forma [1,-1,1,1] con un offset de -0.5, y se entiende que cada bit lo representa con estos cuatro valores, de tal manera que debe formar rectas con pendientes positivas y negativas en cada uno de los bits, como lo es una señal diente de sierra. De esta manera, se intuye que, al querer otro tipo de codificación, se debe cambiar el vector **h** según se requiera.

cod	h	sps	offset
manchester	[1,1,1,1,-1,-1,1,1]	8	-0,5
unipolar rz	[1,1,1,1,0,0,0,0]	8	0
bipolar rz	[1,1,1,1,0,0,0,0]	8	-0,5
polar nrz	[1,1,1,1,1,1,1,1]	8	-0,5

Tabla 2. Codificaciones.

5. Preguntas de control

- Combinación de bloques: La combinación de bloques convierte una secuencia binaria unipolar (0,1) en una secuencia bipolar (-1,1). Para ello, se resta 0.5 a cada

muestra y posteriormente se multiplica por 2.

- b. Interpolation FIR Filter: El parámetro *Interpolation* toma el valor SPS porque cada símbolo debe representarse mediante SPS muestras. Si se modifica este valor, cambia la frecuencia de muestreo y la representación temporal de la señal. En los cambios Se debe conectar un *QT GUI Time Sink* y un *QT GUI Frequency Sink* en el punto p3 para observar la señal en el dominio del tiempo y de la frecuencia. Y para la formula permite conocer el ancho de banda donde R_b es la tasa de bits y SPS las muestras por símbolo. Y para la Frecuencia de muestreo en p3 si se conocen ya que la interpolación incrementa la frecuencia de muestreo en un factor SPS.
- c. Aunque ambas señales se convierten en secuencias binarias bipolares, la distribución de bits es diferente. Por ello, la autocorrelación cambia y la PSD obtenida también es distinta.
- d. Limita la velocidad de procesamiento para evitar que GNU Radio utilice todos los recursos del procesador cuando no existe hardware SDR conectado.
- e. Si no se realiza la conversión a bipolar, aparece una componente DC importante alrededor de 0 Hz, modificando la PSD y concentrando potencia en bajas frecuencias.
- f. Teóricamente el ruido blanco tiene ancho de banda infinito. Sin embargo, en GNU Radio se observa un ancho de banda finito debido a las limitaciones de muestreo y procesamiento del sistema.
- g. No. Aunque teóricamente una señal rectangular tiene ancho de banda infinito, en GNU Radio se observa un ancho de banda limitado debido a la frecuencia de muestreo finita del sistema.
- h. La fórmula para calcular el número total de lóbulos espectrales equivalentes $N_{\text{lóbulos}} = 2 * Sps$
- i. El rango total de frecuencias (span espectral) que ocupa la representación de la señal en GNU Radio se calcula mediante: $\text{Rango Total de Frecuencia} = f_s = R_b * Sps$
- j. La resolución espectral (Δf) define la distancia mínima en frecuencia para diferenciar dos componentes espectrales adyacentes. Se calcula como: $\Delta f = f_s / N$
- k. Al configurar $K = 16$ en el bloque Unpack K Bits, suceden los siguientes efectos: 1. Desempaquetado de 16 bits por palabra: El bloque interpretará cada valor de entrada como una palabra de 16 bits, descomponiéndola en 16 muestras individuales de 1 bit (valores 0 o 1) a la salida. 2. Multiplicación de la Tasa de Muestras (Factor 16x): La frecuencia de muestras a la salida del bloque será 16 veces superior a la frecuencia de entrada.

- l. Para obtener la frecuencia de muestreo a la entrada del bloque Unpack K Bits ($f_{s_in_unpack}$), se sigue este procedimiento: 1. Determinación de la Tasa de Bits (R_b):
El ancho de banda visible en el analizador espectral corresponde a $f_s = BW$. Dado que el número de lóbulos en dicha ventana es $N_{\text{lóbulos}} = f_s / R_b = BW / R_b$, podemos despejar R_b :
 $R_b = BW / N_{\text{lóbulos}}$
- 2. Relación de Desempaquetado del Bloque Unpack:
Cada palabra que ingresa al bloque Unpack K Bits se convierte en K bits de salida. Por ende, la tasa de datos de entrada es K veces menor que la tasa de bits final:
 $f_{s_in_unpack} = R_b / K$
- 3. Fórmula Final Consolidada:
 $f_{s_in_unpack} = BW / (K * N_{\text{lóbulos}})$
- m. La frecuencia de muestreo a la salida del bloque Unpack K Bits ($f_{s_out_unpack}$) se calcula multiplicando la frecuencia de entrada por el parámetro K:
 $f_{s_out_unpack} = K * f_{s_in_unpack}$
Explicación:
- El bloque toma cada muestra de entrada de K bits y la expande serialmente en K muestras individuales de 1 bit.
- Para conservar la tasa de información temporal, por cada segundo ingresan $f_{s_in_unpack}$ muestras y se producen $K * f_{s_in_unpack}$ muestras en la salida.
- n. La frecuencia de muestreo a la salida del bloque Char to Float es exactamente igual a la frecuencia de muestreo a la entrada: $f_{s_out_c2f} = f_{s_in_c2f}$
Explicación Técnica: El bloque Char to Float realiza exclusivamente un cambio de tipo de dato (Type Casting) de entero de 8 bits (char) a formato de punto flotante de 32 bits (float). Dado que no ejecuta operaciones de interpolación ni de decimación, la relación de muestras entre entrada y salida es 1:1, manteniendo inalterada la tasa de muestreo.
- o. La PSD de una señal binaria aleatoria bipolar se asemeja al ruido blanco en la ventana de visualización cuando: $Sps = 1$
- p. Para lograr que los impulsos binarios tomen forma de diente de sierra, se modifica el vector de coeficientes h del bloque Interpolation FIR Filter:
1. Configuración de Coeficientes en h:
En lugar de definir h como un pulso rectangular unitario de longitud Sps:
 $h = [1.0] * Sps$
Se define h como una rampa lineal de longitud Sps (ascendente o descendente):
 $h = [i / (Sps - 1) \text{ for } i \text{ in range}(Sps)]$
O alternativamente usando la librería NumPy en Python:

```
import numpy as np  
h = np.linspace(0, 1, Sps)
```

2. Principio de Funcionamiento:

El bloque Interpolation FIR Filter realiza la convolución discreta de la secuencia de bits sobremuestreada con la respuesta al impulso h . Al configurar h como una rampa de Sps muestras, cada bit se convoluciona con esta rampa, transformando el pulso cuadrado en una forma de onda triangular/diente de sierra en el dominio del tiempo.

CONCLUSIONES

La densidad espectral de potencia (PSD) permitió analizar cómo se distribuye la potencia de las señales en frecuencia, facilitando la comprensión de su comportamiento espectral y su relación con parámetros como la tasa de bits y las muestras por símbolo.

Mediante GNU Radio fue posible observar el efecto de diferentes parámetros del sistema sobre la señal y su PSD, comprobando que el valor de Sps influye directamente en la frecuencia de muestreo y en la representación temporal de la señal.

El análisis de señales aleatorias, ruido blanco y señales reales de audio e imagen permitió relacionar los conceptos teóricos estudiados con aplicaciones prácticas de los sistemas de comunicaciones basados en Radio Definida por Software

6. REFERENCIAS

[1] H. O. Boada and O. M. R. Torres, “Comunicaciones digitales basadas en radio definida por software,” 2019.

[2] L. A. L. C. and D. C. Toloza, “Análisis frecuencial y de la densidad espectral de potencia de la estabilidad de sujetos amputados,” *TecnoLógicas*, vol. 23, pp. 3–18, 2020,[Online].Available:http://www.scielo.org.co/scielo.php?script=sci_arttext&pid=S012377992020000200003&nrm=iso